



Guide to Installing **SFML 2.6.2** for use with **Visual Studio Code** (Windows & Mac - Apple Silicon)

Guide Prepared by: Liam Persad

Disclaimer:

This guide is for general guidance only. The author has made every effort to provide accurate instructions and avoid errors, but it may not cover every scenario and does not guarantee a successful installation or error-free experience. Unexpected issues may occur depending on your system or setup.

Contents:

Section 1: Preparing Visual Studio Code for C++ Development.....	1
1. Installing Visual Studio Code	1
2. Installing a C++ Compiler	1
3. Installing Visual Studio Code Extensions.....	6
Section 2: Installing SFML 2.6.2	8
1. Downloading SFML 2.6.2	8
Section 3: Configuring Visual Studio Code for SFML Projects	12
1. Building the Template Folder Structure.....	12
2. Creating Configuration Files	12
3. Populating Configuration Files	15
Section 4: Running Your First SFML Program.....	20
Section 5: Setting Up SFML Projects for GitHub Collaboration	22
1. Understanding Your Project Structure	22
2. Installing Git on Visual Studio Code (Windows ONLY)	23
3. Setting Up Your Folder for GitHub Collaboration	24
4. Publishing Your GitHub Repository	25
5. Cloning From an Existing Repository	27

Section 1: Preparing Visual Studio Code for C++ Development

1. Installing Visual Studio Code

Download and install the latest version of Visual Studio Code from <https://code.visualstudio.com>.

(At the time of writing, the latest version available is Version 1.104.2)

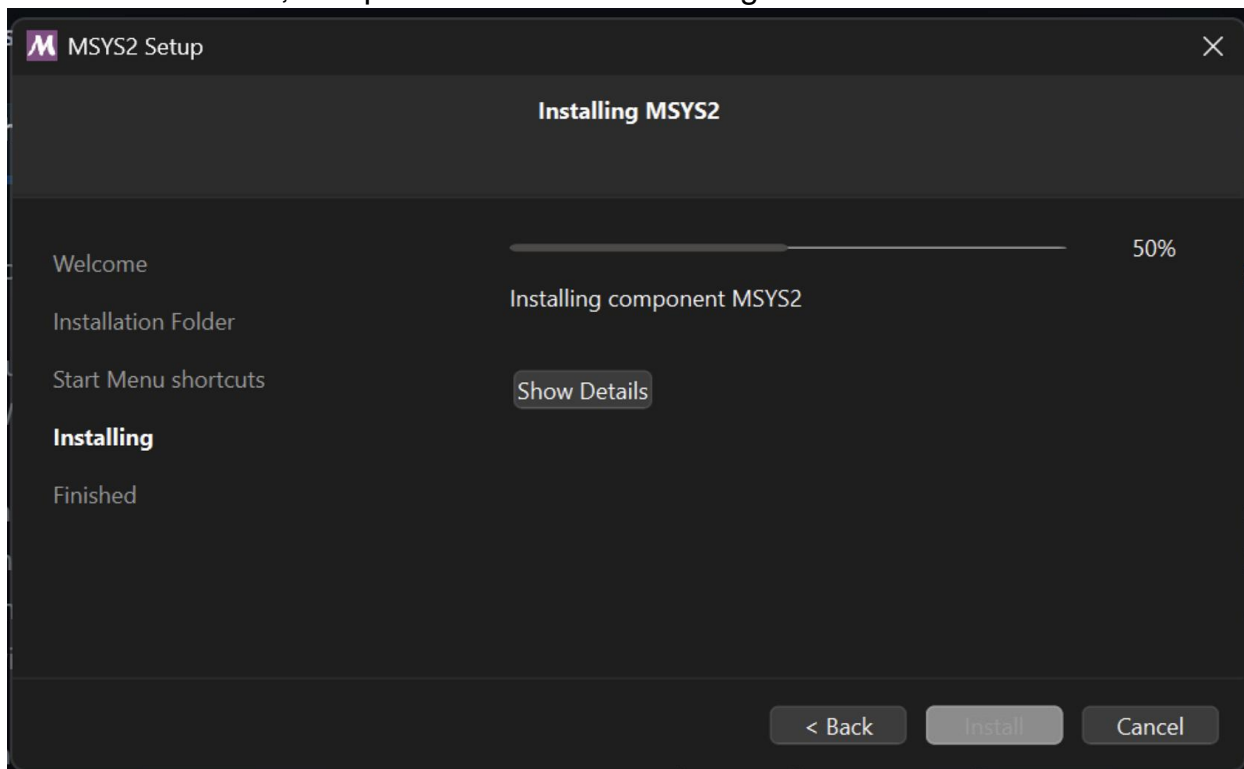
2. Installing a C++ Compiler

Follow the instructions for your respective operating system.

Windows:

Microsoft recommends installing MinGW using 'msys2', which can be downloaded using this link: https://github.com/msys2/msys2-installer/releases/download/2024-12-08/msys2-x86_64-20241208.exe

Once downloaded, complete the installation using the installation wizard.



After installation, the msys2 terminal will open.

Copy, paste and execute the following command into the terminal:

```
pacman -S mingw-w64-x86_64-gcc
```

When prompted, enter 'Y' to accept the installation.

Once complete, do the same with the following command

```
pacman -S mingw-w64-x86_64-gdb
```

When prompted, enter 'Y' to accept the installation.

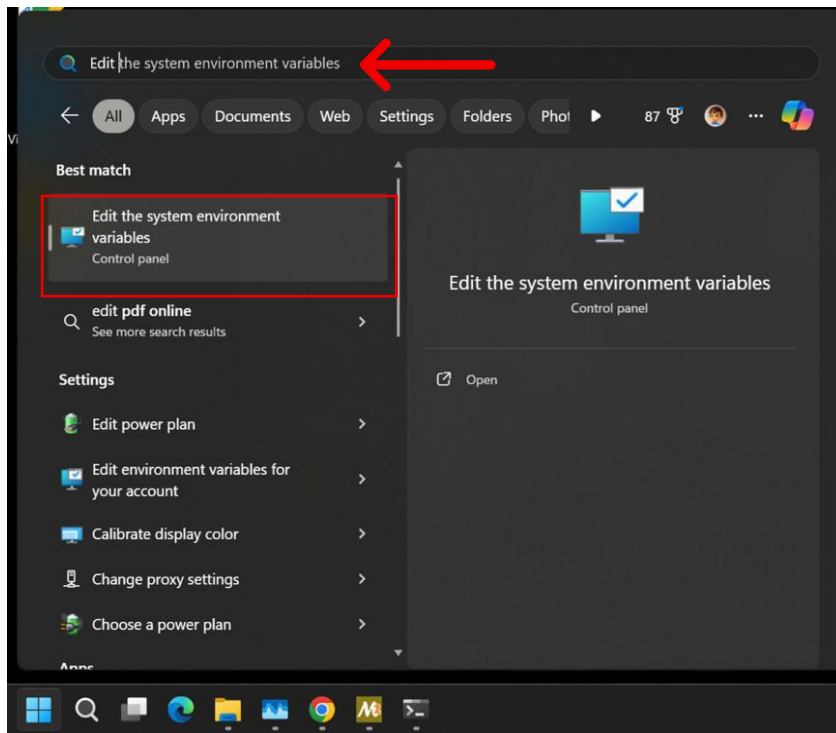
```
liamc@Liam-Surface MSYS ~
$ pacman -S mingw-w64-x86_64-gdb
resolving dependencies...
looking for conflicting packages...

Packages (18) mingw-w64-x86_64-bzip2-1.0.8-3  mingw-w64-x86_64-expat-2.6.4-1
mingw-w64-x86_64-libffi-3.4.6-1  mingw-w64-x86_64-libsystre-1.0.1-6
mingw-w64-x86_64-libtre-0.9.0-1  mingw-w64-x86_64-mpdecimal-4.0.0-1
mingw-w64-x86_64-ncurses-6.5.20240831-1  mingw-w64-x86_64-openssl-3.4.0-1
mingw-w64-x86_64-python-3.12.7-3  mingw-w64-x86_64-readline-8.2.013-1
mingw-w64-x86_64-sqlite3-3.46.1-1  mingw-w64-x86_64-tcl-8.6.13-1
mingw-w64-x86_64-termcap-1.3.1-7  mingw-w64-x86_64-tk-8.6.13-1
mingw-w64-x86_64-tzdata-2024b-1  mingw-w64-x86_64-xxhash-0.8.2-2
mingw-w64-x86_64-xz-5.6.3-3  mingw-w64-x86_64-gdb-15.2-2

Total Download Size: 44.41 MiB
Total Installed Size: 312.54 MiB

:: Proceed with installation? [Y/n] Y
:: Retrieving packages...
mingw-w64-x86_64-gdb-15.2-... 3.8 MiB 2.09 MiB/s 00:02 [#####] 100%
mingw-w64-x86_64-tcl-8.6.1... 2.7 MiB 1327 KiB/s 00:02 [#####] 100%
mingw-w64-x86_64-tk-8.6.13... 2029.2 KiB 974 KiB/s 00:02 [#####] 100%
mingw-w64-x86_64-ncurses-6... 1748.3 KiB 2.08 MiB/s 00:01 [#####] 100%
mingw-w64-x86_64-python-3... 23.1 MiB 7.07 MiB/s 00:03 [#####] 100%
mingw-w64-x86_64-xz-5.6.3-... 759.2 KiB 559 KiB/s 00:01 [#####] 100%
mingw-w64-x86_64-readline-... 411.7 KiB 427 KiB/s 00:01 [#####] 100%
mingw-w64-x86_64-tzdata-20... 197.3 KiB 371 KiB/s 00:01 [#####] 100%
mingw-w64-x86_64-expat-2.6... 164.1 KiB 354 KiB/s 00:00 [#####] 100%
mingw-w64-x86_64-openssl-3... 7.5 MiB 1828 KiB/s 00:04 [#####] 100%
mingw-w64-x86_64-sqlite3-3... 1708.3 KiB 788 KiB/s 00:02 [#####] 100%
mingw-w64-x86_64-mpdecimal... 150.8 KiB 265 KiB/s 00:01 [#####] 100%
mingw-w64-x86_64-libtre-0... 79.5 KiB 151 KiB/s 00:01 [#####] 100%
mingw-w64-x86_64-xxhash-0... 116.4 KiB 137 KiB/s 00:01 [#####] 100%
mingw-w64-x86_64-bzip2-1.0... 90.9 KiB 134 KiB/s 00:01 [#####] 100%
mingw-w64-x86_64-libffi-3... 42.6 KiB 90.5 KiB/s 00:00 [#####] 100%
mingw-w64-x86_64-termcap-1... 27.3 KiB 58.0 KiB/s 00:00 [#####] 100%
mingw-w64-x86_64-libsystre... 9.6 KiB 25.2 KiB/s 00:00 [#####] 100%
Total (18/18) 44.4 MiB 8.14 MiB/s 00:05 [#####] 100%
(18/18) checking keys in keyring [#####] 100%
```

Once the installation is complete, open the 'Start Menu' and begin typing: **Edit the system environment variables** and select the highlighted option.



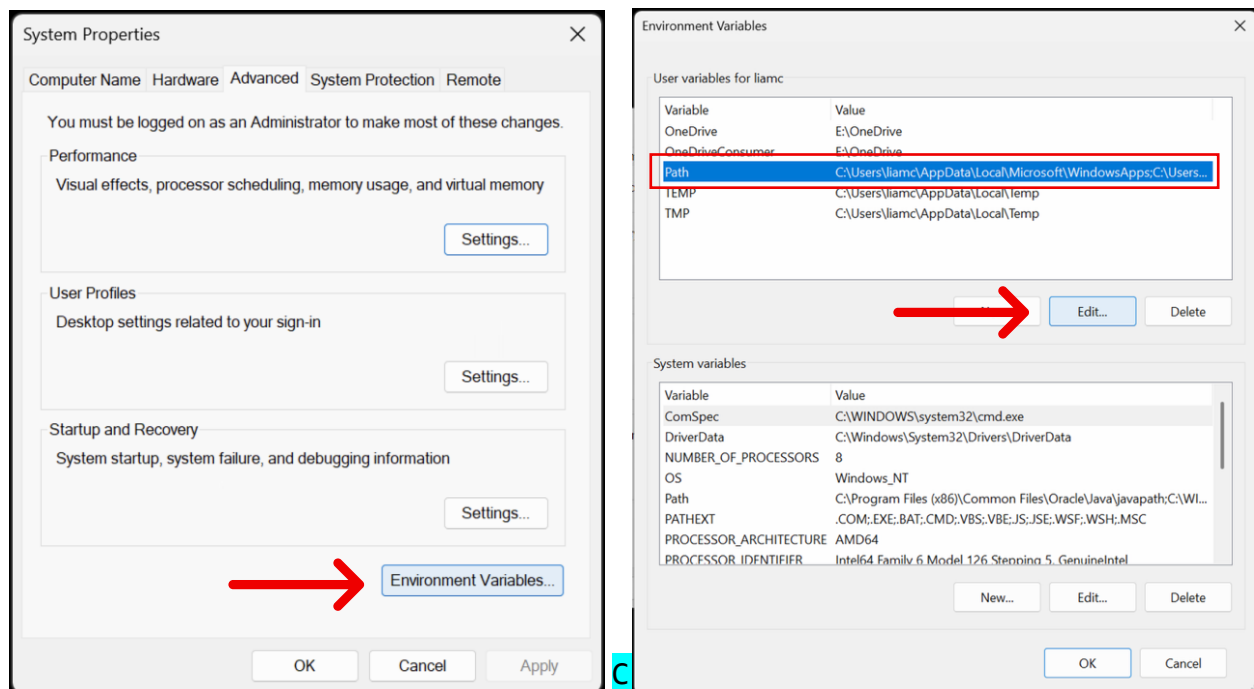
If this option doesn't appear in the Start Menu (sometimes due to language settings), you can navigate the same menu as this:

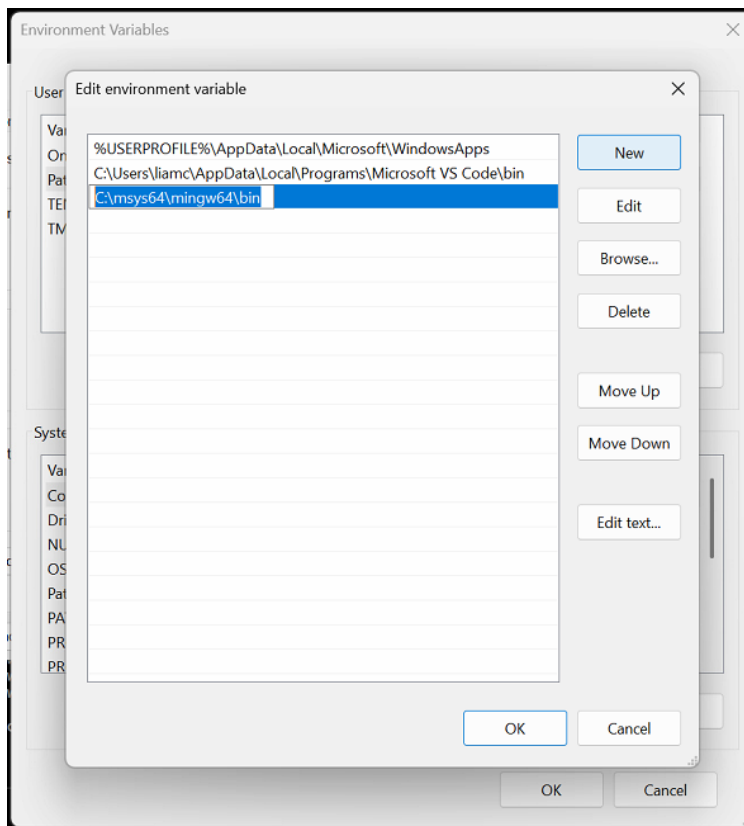
Right-click Start button and select 'System' (システム / 系统).

In the 'System' window, on the right side (or bottom), click 'Advanced system settings'

(システムの詳細設定 / 高级系统设置).

In the 'System Properties' menu that opens, select 'Environment Variables'. Then under 'User variables', select 'Path' and click on the "Edit..." button.





Then select 'OK' to save the path variable.

Once saved, verify your installation. To do this:

1. Open 'Command Prompt' from the Start Menu.
2. Verify the version of your compiler using `gcc --version`
3. Verify the version of your debugger using `gdb --version`

You should see a similar output below:

```
Microsoft Windows [Version 10.0.26100.6584]
(c) Microsoft Corporation. All rights reserved.

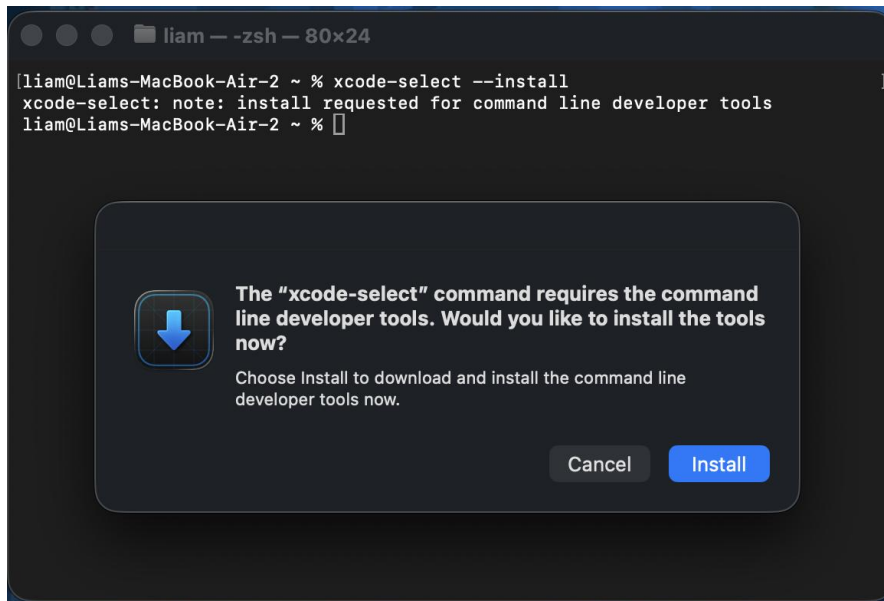
C:\Users\liamc>gcc --version
gcc (Rev2, Built by MSYS2 project) 14.2.0
Copyright (C) 2024 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

C:\Users\liamc>gdb --version
GNU gdb (GDB) 15.2
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
```

Mac (Apple Silicon):

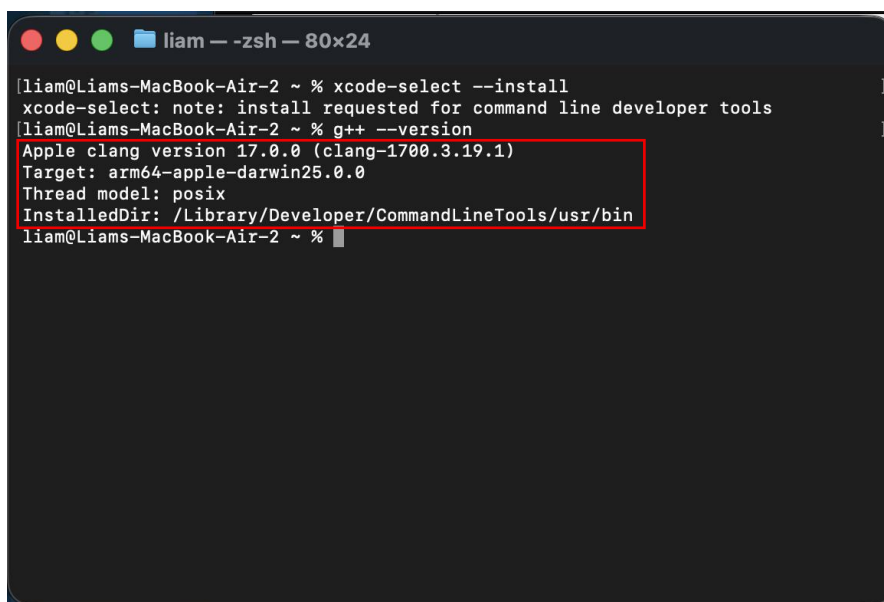
On macOS, open the 'Terminal' and enter `xcode-select --install`

Select the 'Install' button from the pop-up window.



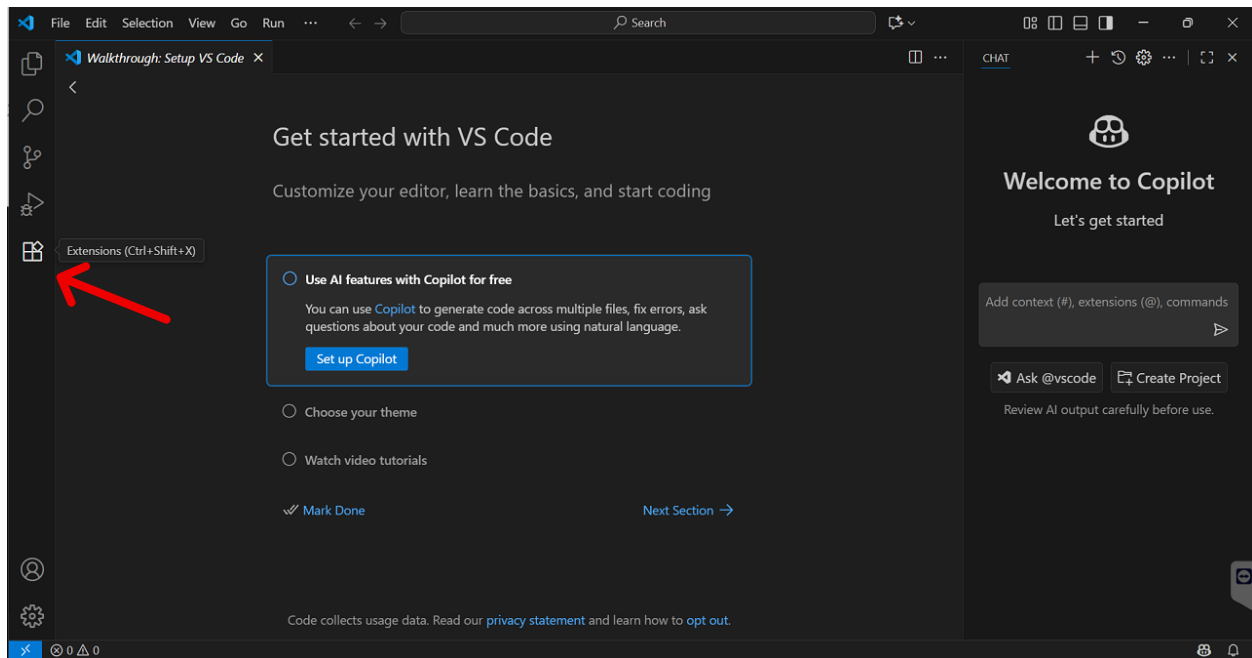
Once the installation is completed, in the terminal, verify your installation by entering `g++ --version`

You should see a similar output below:

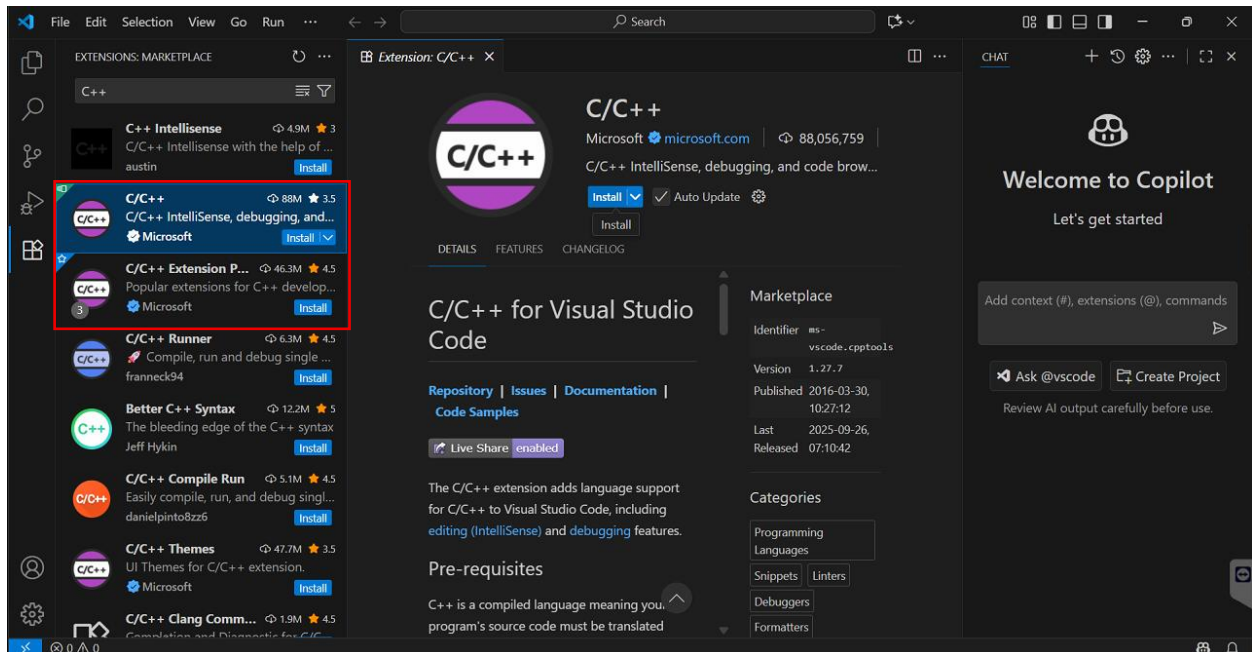


3. Installing Visual Studio Code Extensions

Open Visual Studio Code and select the Extensions icon from the left toolbar.

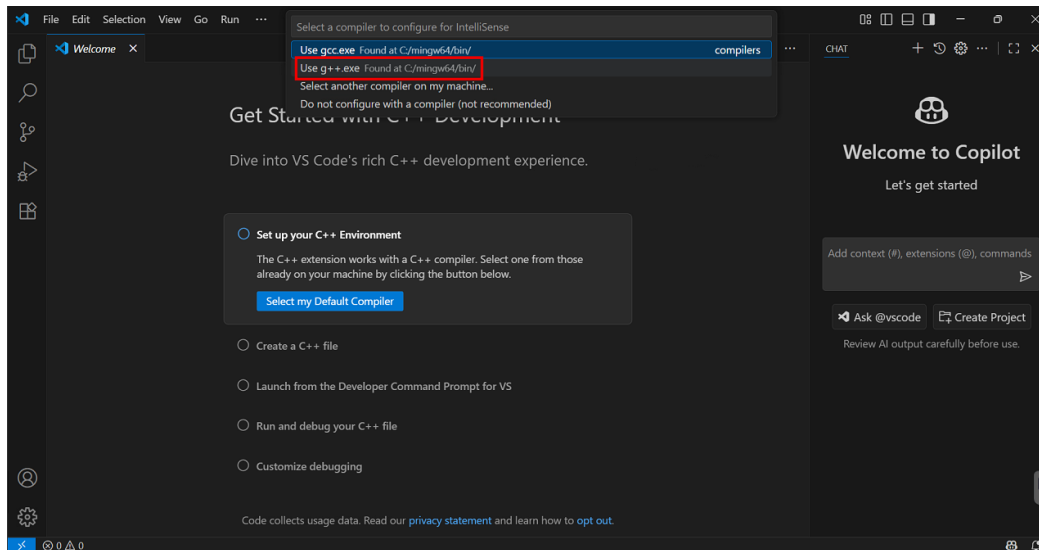


Enter “C++” into the search bar and install both “C/C++” and “C/C++ Extension Pack”.

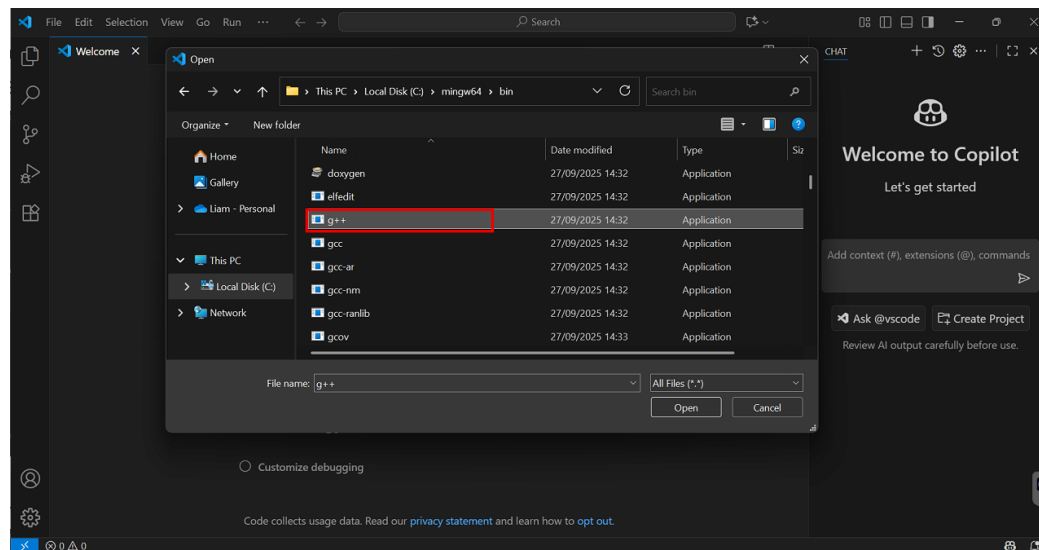


When the installation of the extensions is complete, the C++ setup screen will appear. At this point, you only need to select your default compiler.

Click on the button to open the drop-down menu and select:
“Use g++.exe” (or clang++ on Mac)



(Note: On Windows, a file explorer window may appear after selecting this option. Simply navigate to 'C:/msys64/mingw64/bin' and select 'g++'.)



Visual Studio Code is now ready for C++ development.

Section 2: Installing SFML 2.6.2

1. Downloading SFML 2.6.2

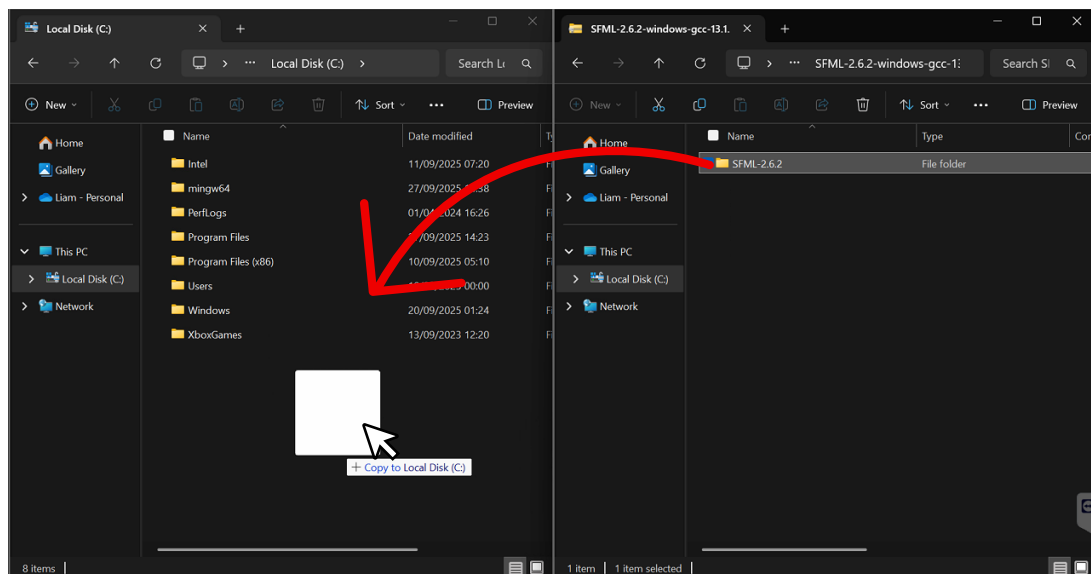
Follow the instructions for your respective operating system.

Windows:

Download and install the 'GCC 13.1.0 MinGW (SEH) - 64-bit' version of SFML 2.6.2 from <https://www.sfml-dev.org/download/sfml/2.6.2/>.



Inside the downloaded ZIP file, copy the 'SFML-2.6.2' folder to the root of your C: drive.



Mac (Apple Silicon):

For simplicity, SFML will be installed using the Homebrew package manager. Go to <https://brew.sh/> and select the copy button next to the installation command.



Paste and run the command in your terminal. You may be required to enter your password to continue. (When entering your password, characters will not show up, but this is normal. Type your password normally and press return/enter.)

You should see a similar output below:

```
liam — -zsh — 80x53

xcode-select: note: install requested for command line developer tools
[liam@Liams-MacBook-Air-2 ~ % g++ --version
Apple clang version 17.0.0 (clang-1700.3.19.1)
Target: arm64-apple-darwin25.0.0
Thread model: posix
InstalledDir: /Library/Developer/CommandLineTools/usr/bin
[liam@Liams-MacBook-Air-2 ~ % /bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
==> Checking for `sudo` access (which may request your password)...
[Password:
==> This script will install:
/opt/homebrew/bin/brew
/opt/homebrew/share/doc/homebrew
/opt/homebrew/share/man/man1/brew.1
/opt/homebrew/share/zsh/site-functions/_brew
/opt/homebrew/etc/bash_completion.d/brew
/opt/homebrew
/etc/paths.d/homebrew

Press RETURN/ENTER to continue or any other key to abort:
```

Once entered, press 'Return / Enter' to proceed with installation.

When the installation is complete, enter `brew --version` to verify your installation.

You should see a similar output below: NEED TO COPY COMMANDS

```
liam — -zsh — 80x22

==> Installation successful!

==> Homebrew has enabled anonymous aggregate formulae and cask analytics.
Read the analytics documentation (and how to opt-out) here:
https://docs.brew.sh/Analytics
No analytics data has been sent yet (nor will any be during this install run).

==> Homebrew is run entirely by unpaid volunteers. Please consider donating:
https://github.com/Homebrew/brew#donations

==> Next steps:
- Run these commands in your terminal to add Homebrew to your PATH:
  echo >> /Users/liam/.zprofile
  echo 'eval "$(/opt/homebrew/bin/brew shellenv)"' >> /Users/liam/.zprofile
  eval "$(/opt/homebrew/bin/brew shellenv)"
- Run brew help to get started
- Further documentation:
  https://docs.brew.sh

liam@Liams-MacBook-Air-2 ~ % brew --version
Homebrew 4.6.14
liam@Liams-MacBook-Air-2 ~ %
```

Then in your terminal, enter `brew info sfml@2` to verify that SFML 2.6.2 is available for download.

You should see a similar output below:

```
liam — -zsh — 80x24

liam@Liams-MacBook-Air-2 ~ % brew info sfml@2
==> sfml@2: stable 2.6.2 (bottled) [keg-only]
Multi-media library with bindings for multiple languages
https://www.sfml-dev.org/

/opt/homebrew/Cellar/sfml@2/2.6.2_1 (954 files, 14.3MB)
  Poured from bottle using the formulae.brew.sh API on 2025-09-10 at 06:02:09
From: https://github.com/Homebrew/homebrew-core/blob/HEAD/Formula/s/sfml@2.rb
License: Zlib

==> Dependencies
Build: cmake ✖, doxygen ✖
Required: flac ✔, freetype ✖, libogg ✔, libvorbis ✔

==> Caveats
sfml@2 is keg-only, which means it was not symlinked into /opt/homebrew,
because this is an alternate version of another formula.

For compilers to find sfml@2 you may need to set:
  export LDFLAGS="-L/opt/homebrew/opt/sfml@2/lib"
  export CPPFLAGS="-I/opt/homebrew/opt/sfml@2/include"

==> Analytics
install: 122 (30 days), 400 (90 days), 1,814 (365 days)
install-on-request: 113 (30 days), 380 (90 days), 1,637 (365 days)
build-error: 0 (30 days)
liam@Liams-MacBook-Air-2 ~ %
```

Now enter `brew install sfml@2` to download and install SFML.

You should see a similar output below:

```
liam — -zsh — 80x10

[liam@Liams-MacBook-Air-2 ~ % brew install sfml@2
==> Fetching downloads for: sfml@2
==> Downloading https://ghcr.io/v2/homebrew/core/sfml/2/manifests/2.6.2_1
Already downloaded: /Users/liam/Library/Caches/Homebrew/downloads/4497621b82de6c5ced1b0fe872d2fa2d73623bbc0af233090285964029cd2350--sfml@2-2.6.2_1.bottle_manifest.json
==> Downloading https://ghcr.io/v2/homebrew/core/sfml/2/manifests/2.6.2_1
##### 100.0%
==> Fetching dependencies for sfml@2: libogg, flac, libpng, freetype and libvorbis
```

Once complete, you can enter `brew info sfml@2` to verify that SFML 2.6.2 is installed correctly.

```
liam — -zsh — 80x24

[liam@Liams-MacBook-Air-2 ~ % brew info sfml@2
==> sfml@2: stable 2.6.2 (bottled) [keg-only]
Multi-media library with bindings for multiple languages
https://www.sfml-dev.org/
Installed
/opt/homebrew/Cellar/sfml@2/2.6.2_1 (916 files, 13.8MB)
Poured from bottle using the formulae.brew.sh API on 2025-09-27 at 16:06:07
From: https://github.com/Homebrew/homebrew-core/blob/HEAD/Formula/s/sfml@2.rb
License: Zlib
==> Dependencies
Build: cmake x, doxygen x
Required: flac ✓, freetype ✓, libogg ✓, libvorbis ✓
==> Caveats
sfml@2 is keg-only, which means it was not symlinked into /opt/homebrew,
because this is an alternate version of another formula.

For compilers to find sfml@2 you may need to set:
  export LDFLAGS="-L/opt/homebrew/opt/sfml@2/lib"
  export CPPFLAGS="-I/opt/homebrew/opt/sfml@2/include"
==> Analytics
install: 122 (30 days), 400 (90 days), 1,814 (365 days)
install-on-request: 113 (30 days), 380 (90 days), 1,637 (365 days)
build-error: 0 (30 days)
liam@Liams-MacBook-Air-2 ~ %
```

SFML has now been installed on your computer.

Section 3: Configuring Visual Studio Code for SFML Projects

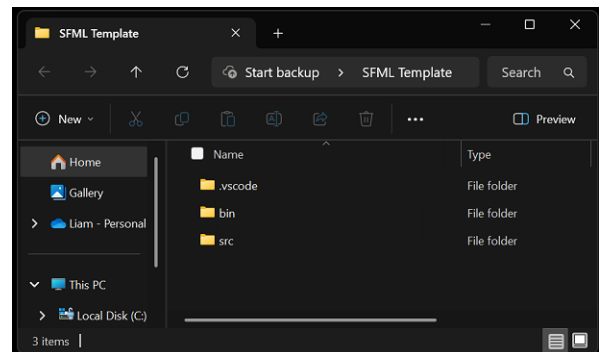
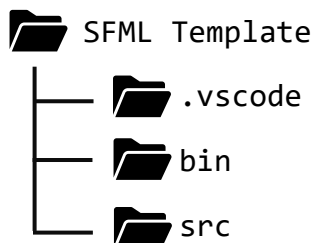
Now that Visual Studio Code is configured for C++ development, and SFML is installed on your computer, we must now tell Visual Studio Code how to build projects using the SFML libraries.

1. Building the Template Folder Structure

For the sake of this guide, we will create a template project folder that can be reused for new SFML projects.

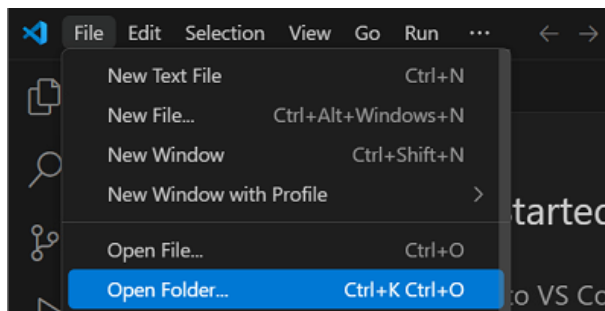
1. Create a new folder and name it “SFML Template” or whatever you like.
2. Within the new folder, create three new folders
 - a. `.vscode` DOES NOT WORK ON MAC
 - b. `bin`
 - c. `src`

Your folder structure should look like this:

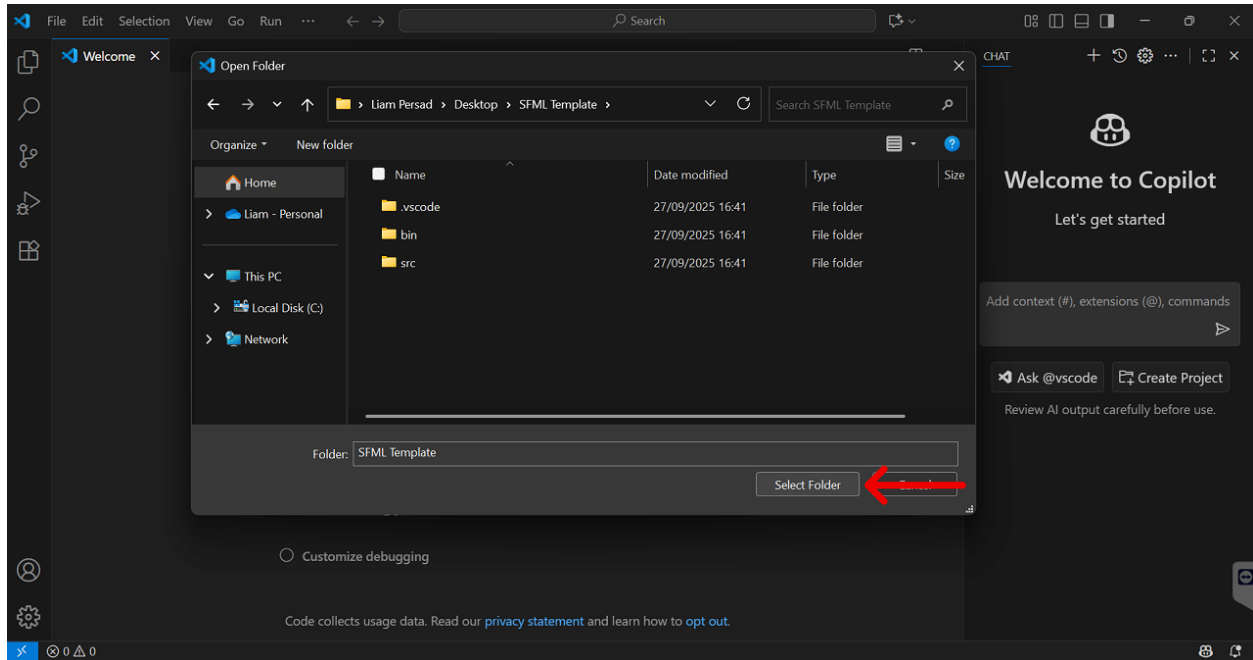


2. Creating Configuration Files

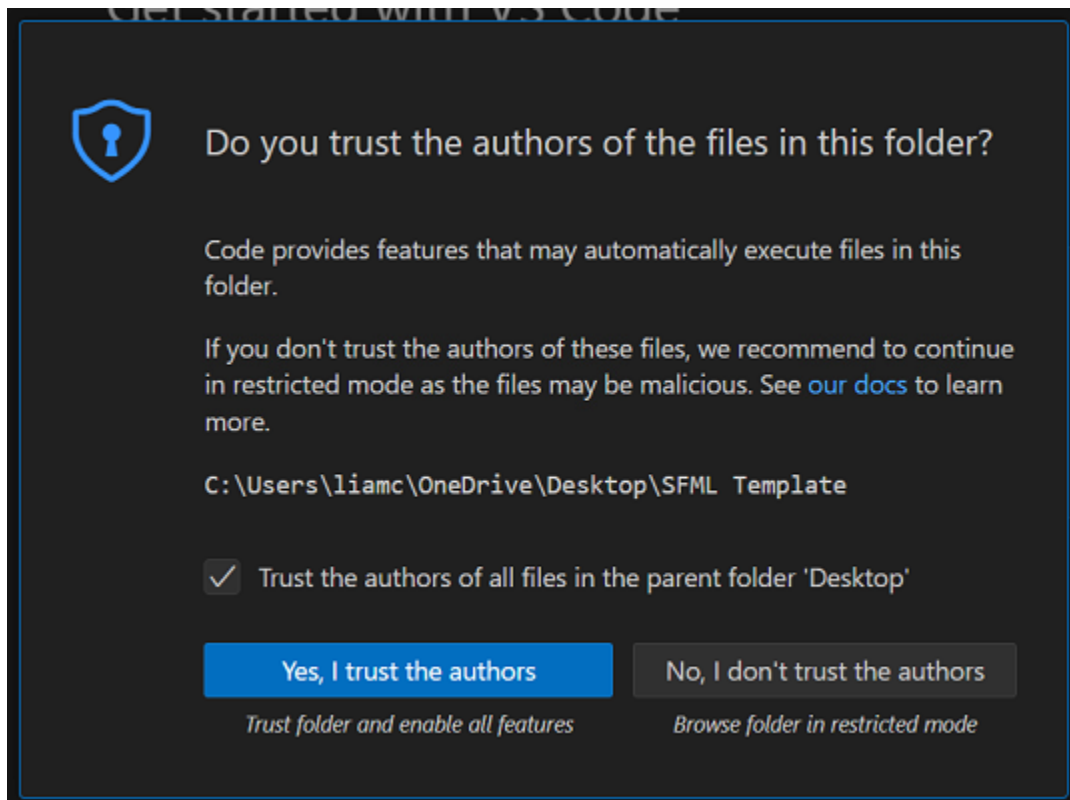
Open Visual Studio Code and select 'File' then 'Open Folder'.



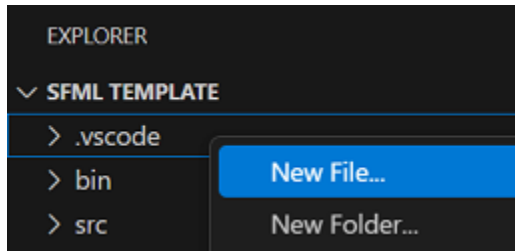
Then find and select the SFML Template folder you created.



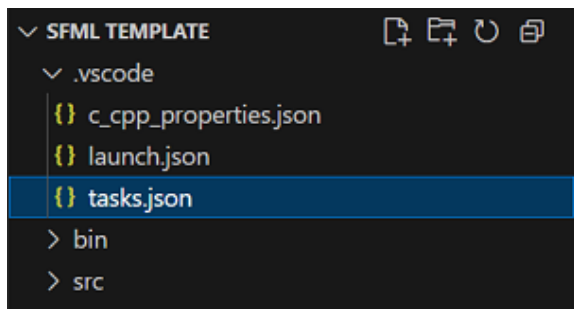
Select “Yes, I trust the authors” if prompted.



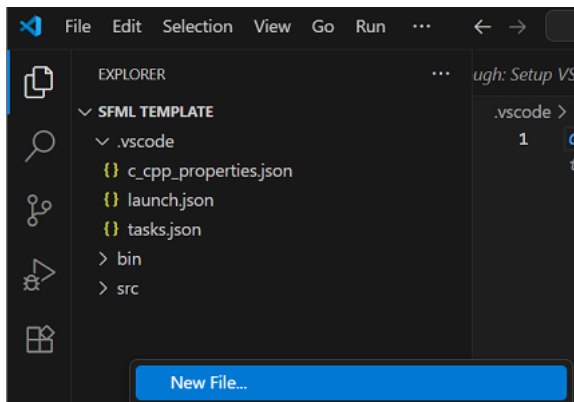
On the left side, right-click on '.vscode' and select "New File".
Name this new file `c_cpp_properties.json`.



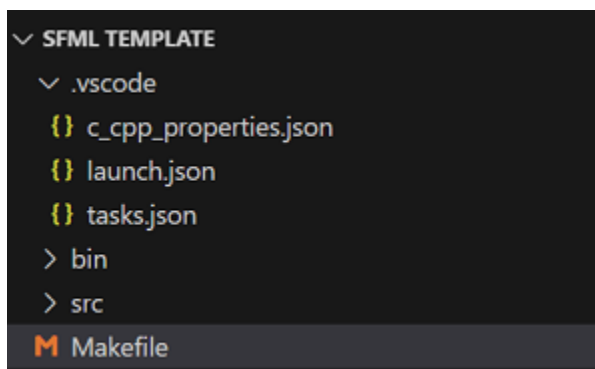
Repeat this process to also create two other files named: `launch.json` and `tasks.json`.



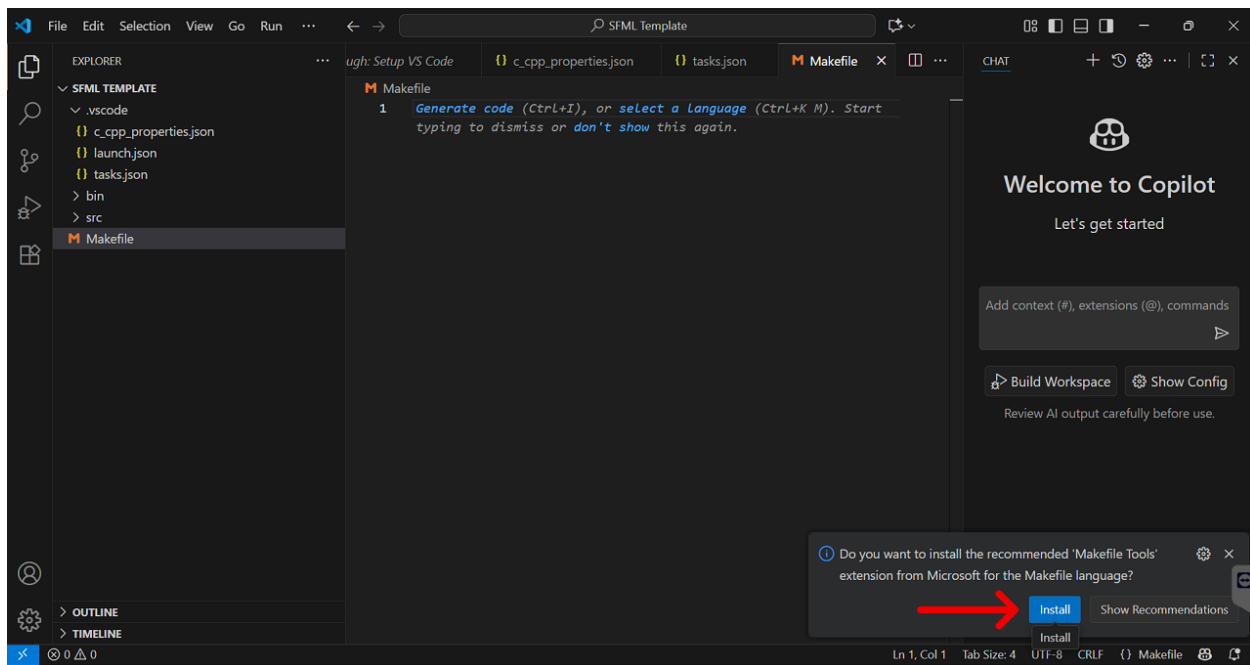
Then right-click in the empty space under 'src' and select "New File".



Name this new file `Makefile`.



Also install the 'Makefile Tools' when prompted.



3. Populating Configuration Files

Follow the instructions for your respective operating system.

Windows:

Copy and paste the configurations for each file as shown below:

c_cpp_properties.json

```
{
  "configurations": [
    {
      "name": "windows-gcc-x64",
      "includePath": [
        "${workspaceFolder}/**",
        "C:/SFML-2.6.2/include" // SFML Installation Path (include)
      ],
      "compilerPath": "C:/msys64/mingw64/bin/g++.exe", // Compiler Installation Path (g++)
      "cStandard": "c17",
      "cppStandard": "c++17",
      "intelliSenseMode": "windows-gcc-x64",
      "defines": [],
      "compilerArgs": [
        "-Wall",
        "-Wextra"
      ]
    }
  ],
  "version": 4
}
```

launch.json

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Launch SFML App",
      "type": "cppdbg",
      "request": "launch",
      "program": "${workspaceFolder}/bin/main.exe",
      "args": [],
      "stopAtEntry": false,
      "cwd": "${workspaceFolder}/src",
      "externalConsole": true,
      "MIMode": "gdb",
      "miDebuggerPath": "C:/msys64/mingw64/bin/gdb.exe", // Compiler Installation Path (gdb)
      "preLaunchTask": "Build C++ program with SFML",
      "setupCommands": [
        {
          "description": "Enable pretty-printing for gdb",
          "text": "-enable-pretty-printing",
          "ignoreFailures": true
        }
      ]
    }
  ]
}
```

tasks.json

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "Build C++ program with SFML",
      "type": "shell",
      "command": "g++",
      "args": [
        "-g",
        "src/*.cpp",
        "-I", "C:/SFML-2.6.2/include", // SFML Installation Path (include)
        "-L", "C:/SFML-2.6.2/lib", // SFML Installation Path (lib)
        "-lsfml-graphics",
        "-lsfml-window",
        "-lsfml-system",
        "-lsfml-audio",
        "-lsfml-network",
        "-std=c++17",
        "-o", "${workspaceFolder}/bin/main.exe"
      ],
      "options": {
        "cwd": "${workspaceFolder}",
        "shell": {
          "executable": "C:/msys64/usr/bin/bash.exe",
          "args": ["-c"]
        }
      },
      "problemMatcher": ["$gcc"],
      "group": {
        "kind": "build",
        "isDefault": true
      },
      "detail": "Build all cpp files for SFML"
    }
  ]
}
```

... continued on next page ...

Makefile

```
# Path to SFML installed
SFML_PATH = C:/SFML-2.6.2

# Automatically find all .cpp files in src/
CPPFILES := $(shell find ./src -type f -name "*.cpp")

# Compiler and flags
CXX = g++
CXXFLAGS = -std=c++17 -g -I$(SFML_PATH)/include
LD_FLAGS = -L$(SFML_PATH)/lib -lsfml-graphics -lsfml-window -lsfml-system -lsfml-audio -lsfml-network

# Output binary
OUTPUT = bin/main.exe

# Default target
all: $(OUTPUT)

$(OUTPUT): $(CPPFILES)
    mkdir -p bin
    $(CXX) $(CPPFILES) $(CXXFLAGS) -o $(OUTPUT) $(LD_FLAGS)

clean:
    rm -f $(OUTPUT)
```

(Note: If you have installed your compiler or SFML library in a location that is different from which is specified in this guide, please alter the **SFML Paths** and **Compiler Paths** as highlighted above.)

Then, ensure you save each file.

Mac (Apple Silicon):

Copy and paste the configurations for each file as shown below:

c_cpp_properties.json

```
{
  "configurations": [
    {
      "name": "Mac",
      "includePath": [
        "${workspaceFolder}/**",
        /opt/homebrew/Cellar/sfml@2/2.6.2\_1/include // sometimes
        /usr/local/cellar/sfml@2...etc
      ],
      "defines": [],
      "cStandard": "c17",
      "cppStandard": "c++14",
      "compilerPath": "/usr/bin/clang++",
      "intelliSenseMode": "macos-clang-arm64",
      "macFrameworkPath": [
        "/System/Library/Frameworks",
        "/Library/Frameworks"
      ]
    }
  ],
  "version": 4
}
```

launch.json

```
{
  "version": "0.2.0",
  "configurations": [
    {
      "name": "Launch SFML App",
      "type": "cppdbg",
      "request": "launch",
      "program": "${workspaceFolder}/bin/main",
      "args": [],
      "stopAtEntry": false,
      "cwd": "${workspaceFolder}/src",
      "environment": [],
      "externalConsole": false,
      "MIMode": "lldb",
      "targetArchitecture": "arm64",
      "preLaunchTask": "Build SFML Project",
      "setupCommands": [
        {
          "description": "Enable pretty-printing for gdb/lldb",
          "text": "-enable-pretty-printing",
          "ignoreFailures": true
        }
      ]
    }
  ]
}
```

... continued on next page ...

tasks.json

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "Build SFML Project",
      "type": "shell",
      "command": "make",
      "options": {
        "cwd": "${workspaceFolder}"
      },
      "group": {
        "kind": "build",
        "isDefault": true
      }
    }
  ]
}
```

Makefile

```
# Path to SFML installed via Homebrew
SFML_PATH = /opt/homebrew/Cellar/sfml@2/2.6.2_1

# Automatically find all .cpp files in src/
cppFileNames := $(shell find ./src -type f -name "*.cpp")

# Compiler and flags
CXX = clang++
CXXFLAGS = -std=c++14 -g -I$(SFML_PATH)/include
LDLAGS = -L$(SFML_PATH)/lib -lsfml-graphics -lsfml-window -lsfml-system -lsfml-audio -lsfml-network

# Default target
all: compile

compile:
    mkdir -p bin
    $(CXX) $(cppFileNames) $(CXXFLAGS) -o bin/main $(LDLAGS)

clean:
    rm -rf bin/main
```

Then, ensure you save each file.

Section 4: Running Your First SFML Program

In your 'src' folder, create a file called **main.cpp** and paste the following sample SFML code into that file.

main.cpp

```
#include <SFML/Graphics.hpp>

int main()
{
    sf::RenderWindow window(sf::VideoMode(200, 200), "SFML works!");
    sf::CircleShape shape(100.f);
    shape.setFillColor(sf::Color::Green);

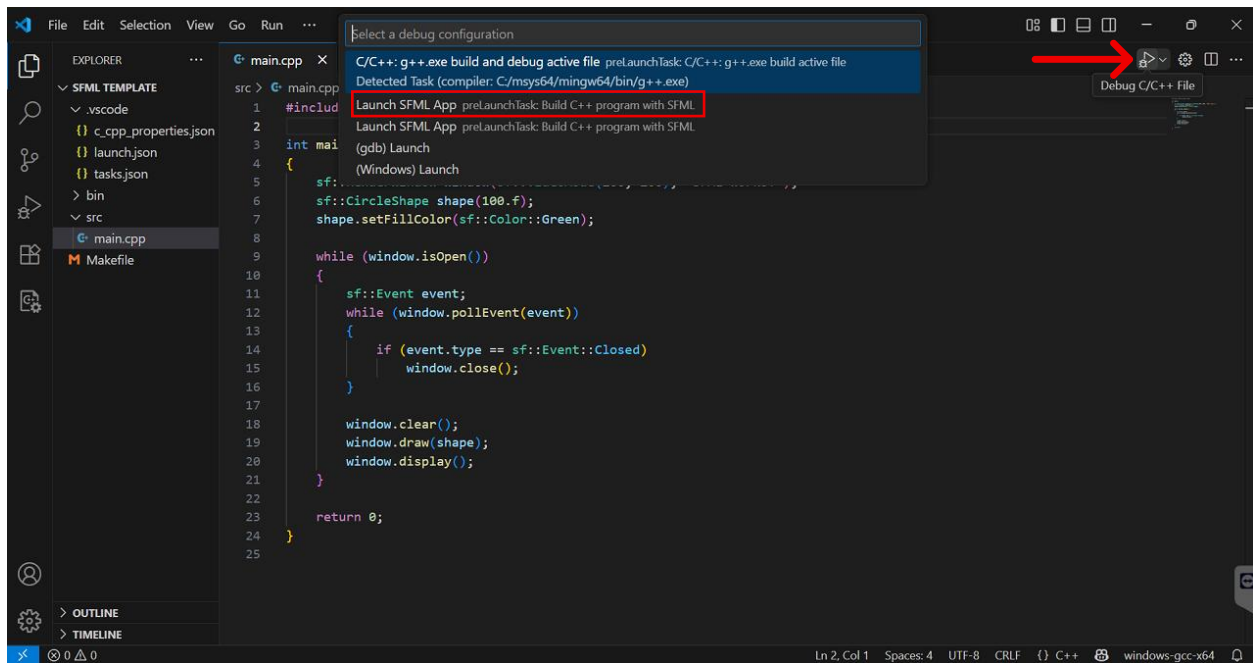
    while (window.isOpen())
    {
        sf::Event event;
        while (window.pollEvent(event))
        {
            if (event.type == sf::Event::Closed)
                window.close();
        }

        window.clear();
        window.draw(shape);
        window.display();
    }

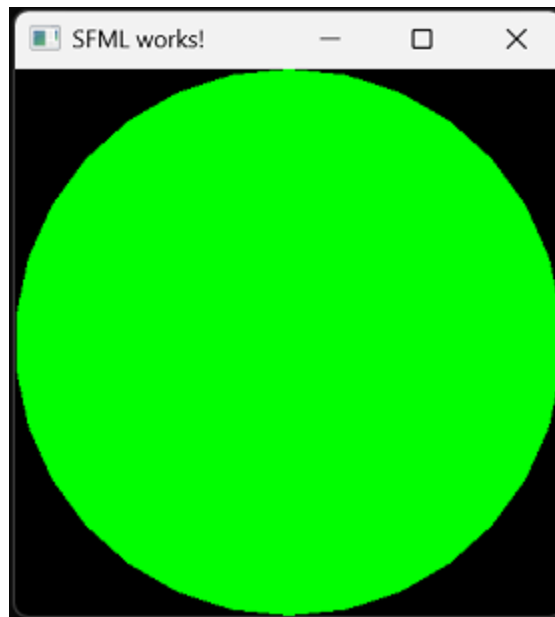
    return 0;
}
```

Then, select the 'Run/Debug' icon on the top-right of your window.

When the drop down menu appears, select **Launch SFML App**.



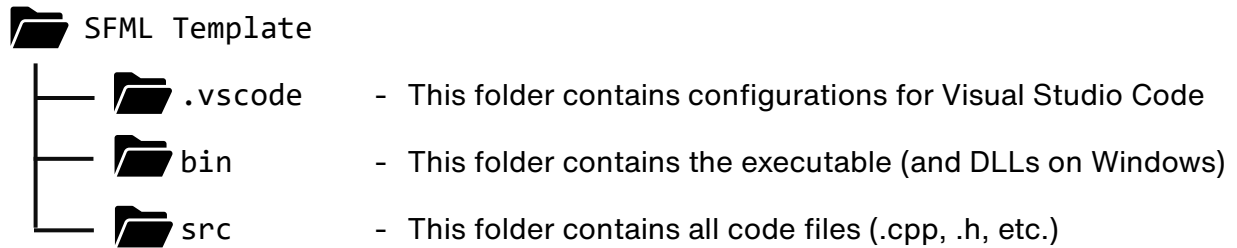
Your C++ code will now be compiled and your SFML window will appear!



Section 5: Setting Up SFML Projects for GitHub Collaboration

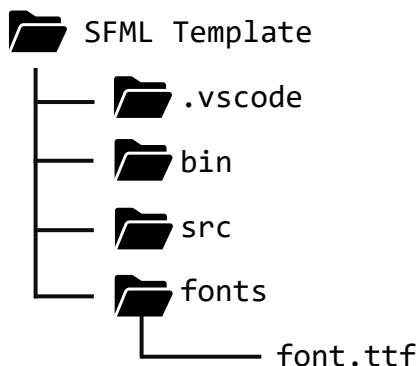
1. Understanding Your Project Structure

From the guide, your folder structure should look like this:



When adding additional assets to your projects, such as fonts, images, etc., these should be placed either in the 'SFML Template' folder or preferably in a dedicated folder.

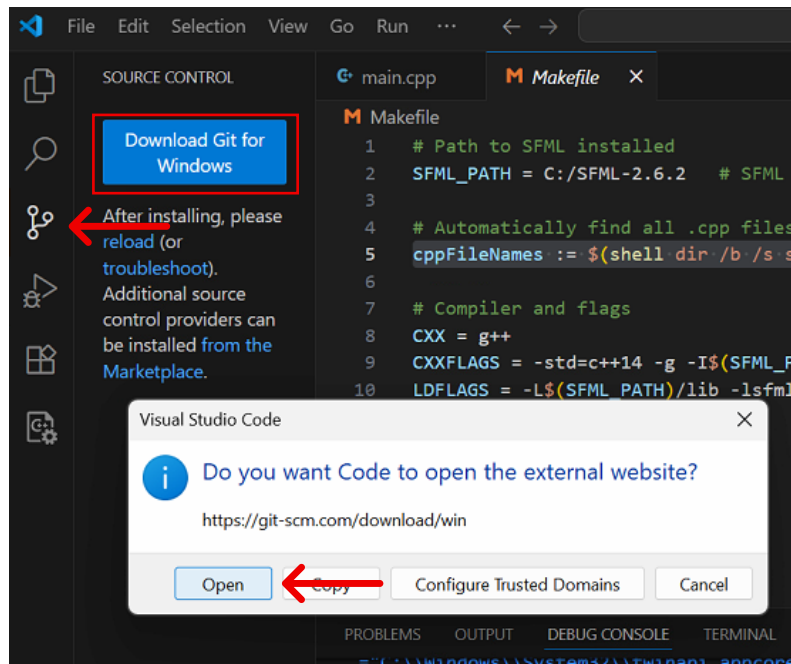
For example, if you were to include a font file with your project, your folder structure should look like this:



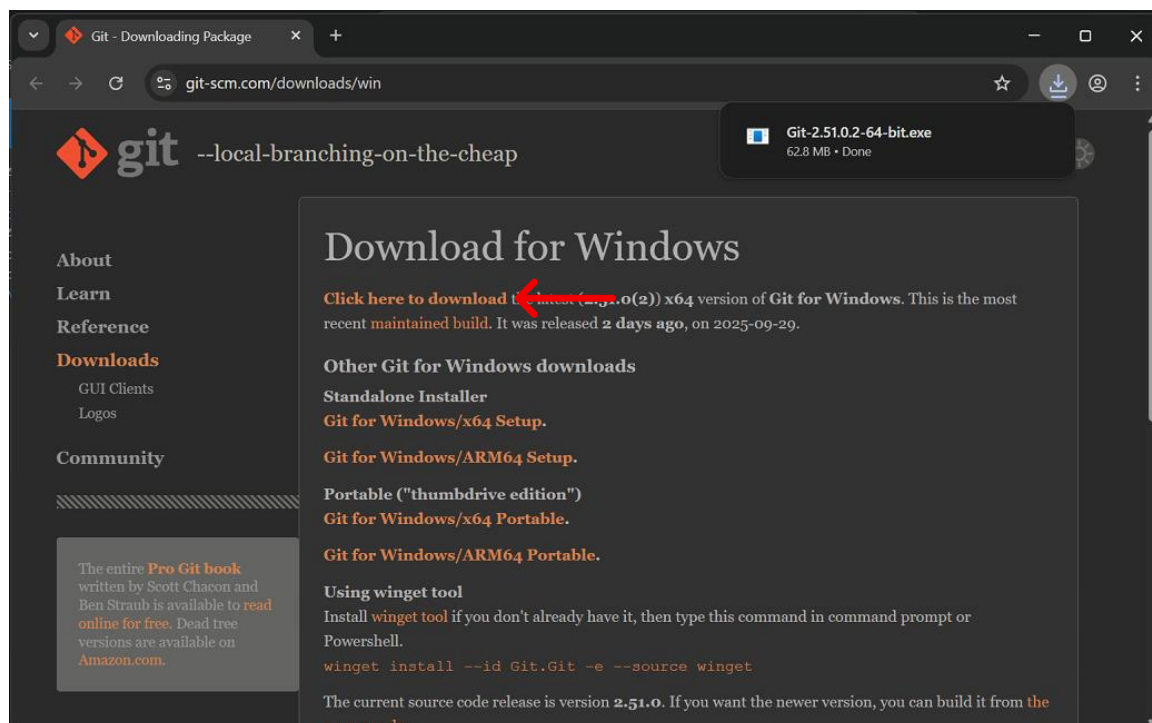
This is done to allow Visual Studio Code to correctly find your additional files when compiling your project.

2. Installing Git on Visual Studio Code (Windows ONLY)

To use GitHub source control integrated into Visual Studio Code, select the 'Source Control' icon on the left toolbar and select 'Download Git for Windows'.



Download and install the latest version of Git from the website using the default settings in the installer. Then close and reopen Visual Studio Code.

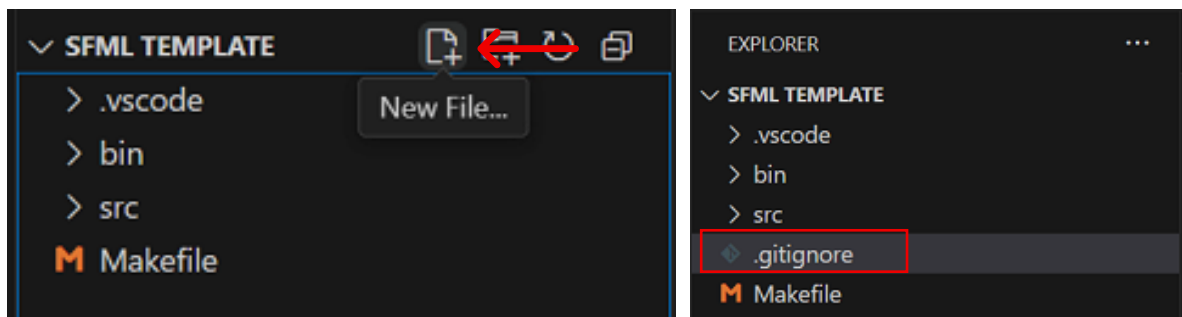


3. Setting Up Your Folder for GitHub Collaboration

The key to creating a GitHub repository that works for all members of a group without being affected by system specific files (such as those we setup earlier) is to create a `.gitignore` file.

This file will only upload the files necessary for members of your project to work on, without affecting your local Visual Studio project setup.

To create this file, click on the 'New File' button next to 'SFML Template' in your 'Explorer' side menu and name the file `.gitignore`.



Then enter the following text into your newly created file and save it:

`.gitignore`

```
/*  
!  
!.gitignore  
!/src/
```

- This line indicates to ignore all files
- These lines indicate to make an exception for the `/src` folder (as this code contains your project code) and the `.gitignore` file.

In the example discussed above, if there was another folder present, such as one for fonts or other assets, those should be included in this file as well, such as shown below.

`.gitignore`

```
/*  
!  
!.gitignore  
!/src/  
!/fonts/
```

4. Publishing Your GitHub Repository

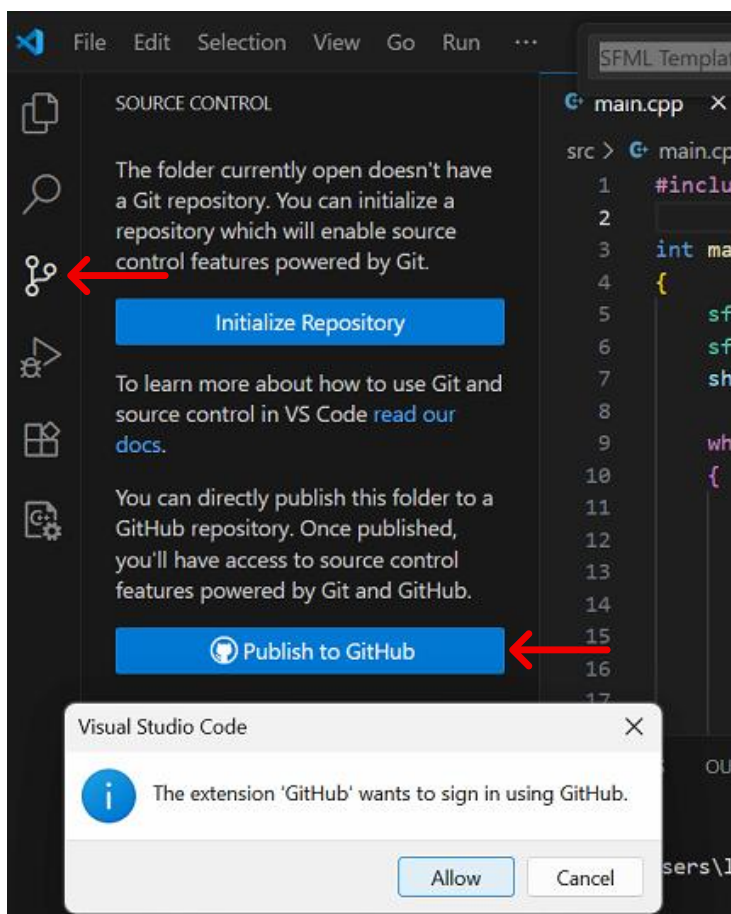
If you haven't already, you need to configure your initial Git username and email.

To do this, open the terminal in Visual Studio Code and enter the following commands (replacing the respective fields with your account information).

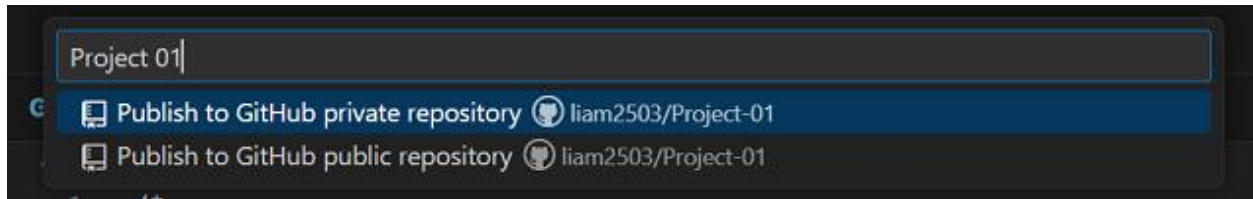
```
git config --global user.name "Your Name"
```

```
git config --global user.email "your@email.com"
```

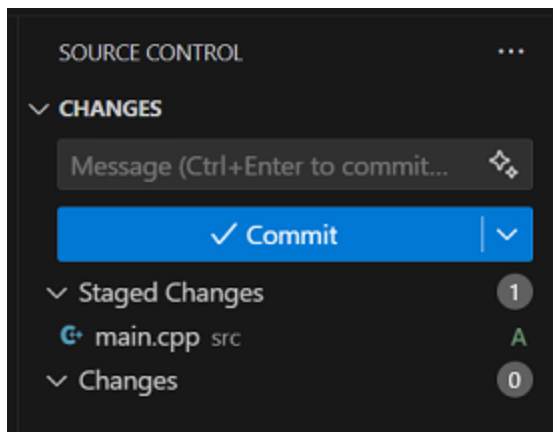
Once completed, elect the 'Source Control' icon from the left toolbar and then select 'Publish to GitHub'. Sign In to GitHub when prompted or create an account if you haven't already.



Then select what you would like to name your repository and whether you would like to publish to a 'Public' or 'Private' repository.

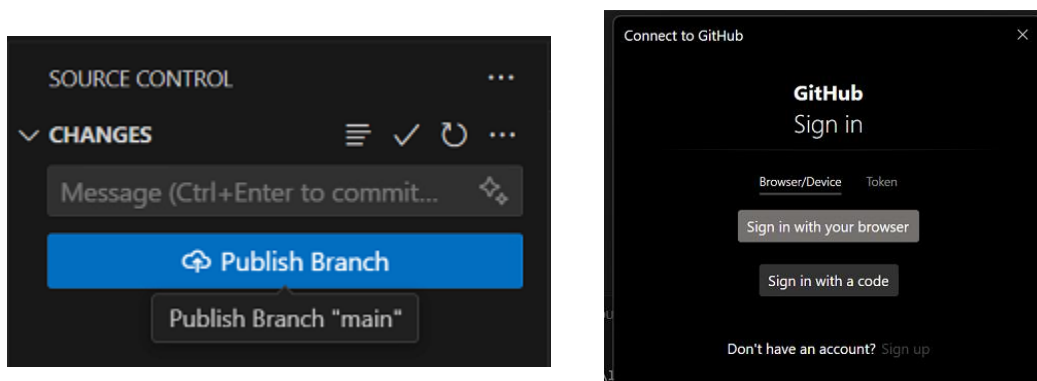


You should then see a summary of all files that will be uploaded to your repository.



Type a brief description of your changes to your code into the 'message' field and then select 'Commit'.

Then select "Publish Branch" to synchronize your project with the GitHub repository. (You may be prompted to Sign In again, but this is only a one-time process).

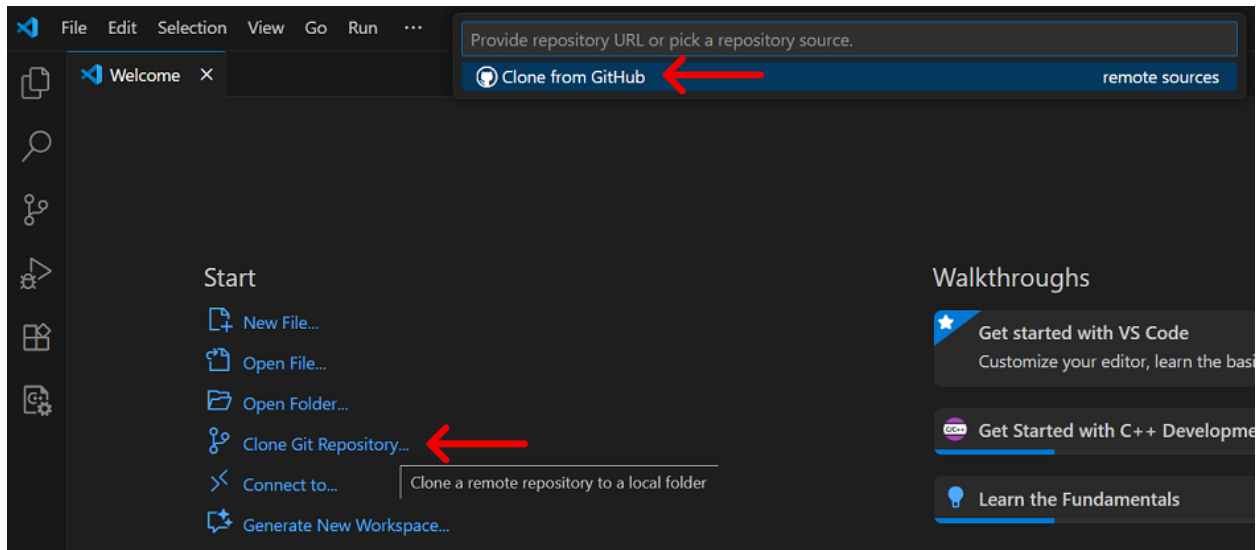


You have now successfully published your SFML project to GitHub.

5. Cloning From an Existing Repository

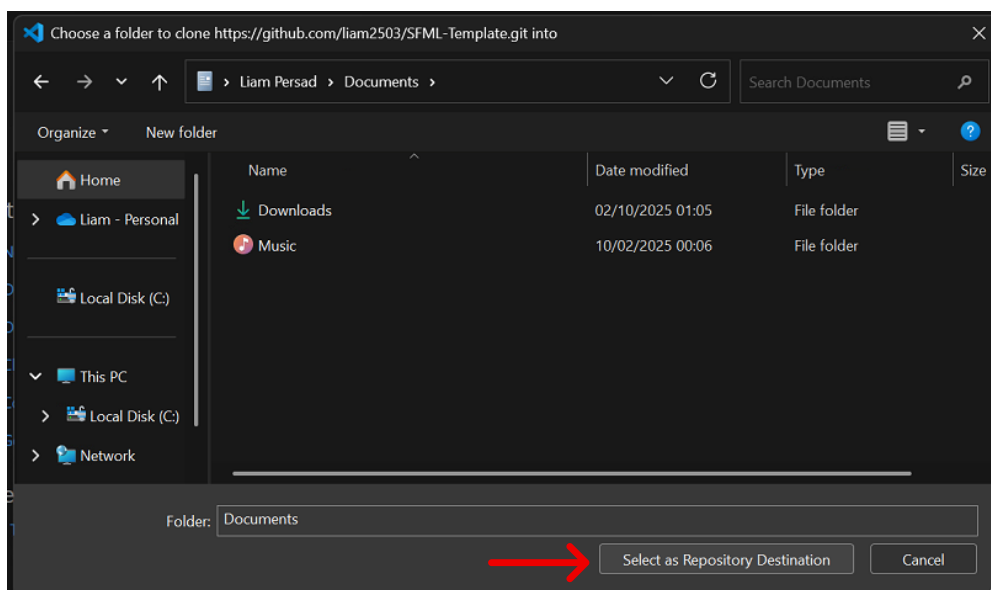
If you are cloning another repository setup by someone else using this guide, follow these steps.

On the Visual Studio Code 'Welcome Screen', select 'Clone Git Repository...' and then select 'Clone from GitHub'.

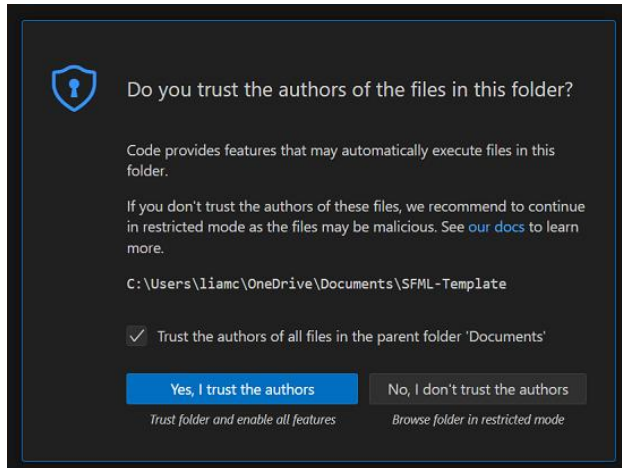


A list of available repositories will appear. Select the repository you want to clone and choose a location on your computer to save it.

(Note: If you have not already been invited to and accepted your invitation to collaborate on a repository, it will not show up here. You must do this before trying to clone.)

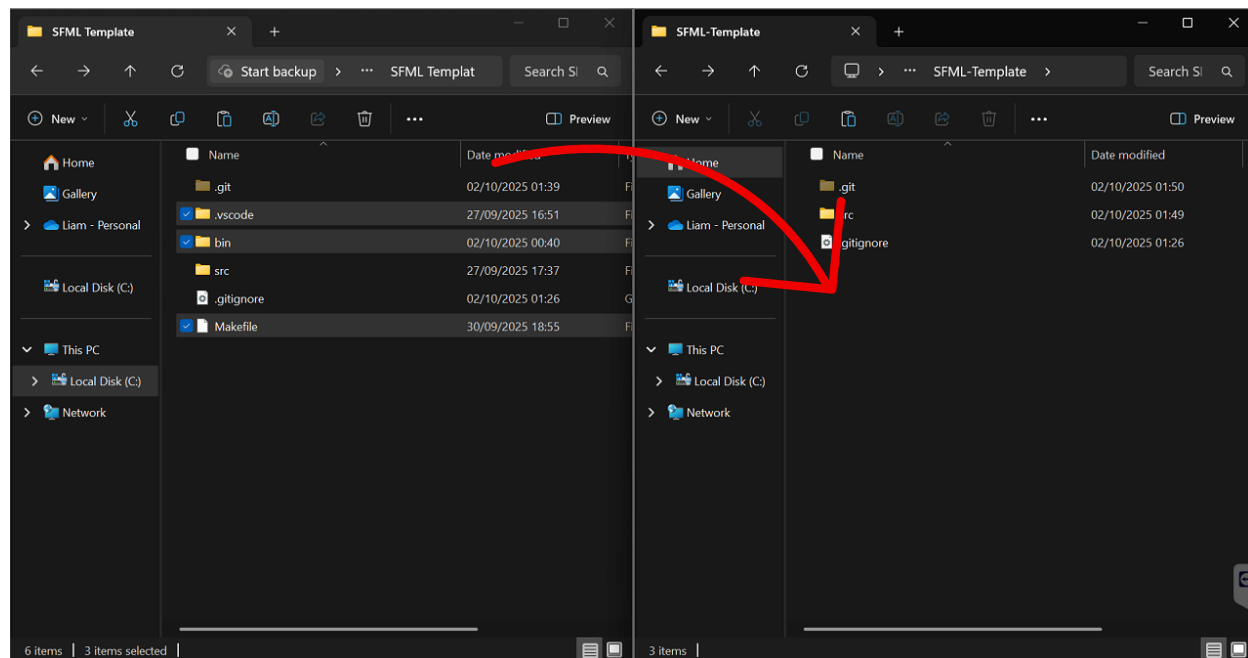


When prompted, open the repository and select 'Yes, I trust the authors.'



Then, from the 'SFML Template' folder previously created, copy (.vscode, bin and Makefile) into the cloned repository folder.

(On Mac, the .vscode folder may be hidden. Press "Command" + "Shift" + "." to show hidden folders.)



Your repo is now ready to be run in Visual Studio Code.

(In rare cases, the name of your project folder may cause issues when compiling your code. Simply close Visual Studio Code, rename your folder and then reopen Visual Studio code and try compiling again.)

References / Credits:

- YouTube. *Installing SFML on Mac*. Available at:
<https://www.youtube.com/watch?v=4UaqmTvq2zk>
- SFML-Dev.org. *Getting Started with Code::Blocks*. Available at:
<https://www.sfm-dev.org/tutorials/2.6/start-cb.php>
- Beatzoid. *sfml-macos* [GitHub repository]. Available at:
<https://github.com/Beatzoid/sfml-macos>
- Stack Overflow. *How do I use SFML on Apple M1 Mac?* Available at:
<https://stackoverflow.com/questions/69720807/how-do-i-use-sfml-on-apple-m1-mac/73402250#73402250>